R2-A2-S8 Tarea documento: Informe de resultados de tratamiento de imagen

Edward Alexander Moreno Matallana

Universidad de Cundinamarca UCUNDINAMARCA

Especialización en Inteligencia Artificial

Programa

2026

Tabla de Contenido

Introducción 2

Objetivos 3

Objetivo General 3

Informe de resultados de tratamiento de imagenes 4

Diagnóstico y comparación de métricas 4

Comparación JPG y PNG 5

Tratamiento de la imagen problemática - Brillo y Contraste 6

Resultados cuantitativos antes y después 7

Tratamiento de la imagen problemática - Histograma 7

Tratamiento de la imagen problemática - Detección del objeto 8

Respuestas a las preguntas orientadoras 11

¿Qué características de una imagen pueden afectar un sistema de percepción computacional? 11

¿Una imagen de mayor resolución necesariamente es mejor para un modelo? 11

¿Qué efecto tuvo la iluminación sobre los datos obtenidos? 11

¿Qué ocurrió con la imagen desenfocada y cómo se reflejó en la medida de nitidez? 11

¿Todos los problemas encontrados pueden corregirse mediante procesamiento? 11

¿Por qué es importante analizar una imagen antes de utilizarla como entrada de un modelo? .. 11

¿El tratamiento aplicado mejoró realmente la imagen para el propósito planteado? 11

Anexo 11

Conclusiones 11

Introducción

El tratamiento digital de imágenes es un proceso fundamental en el área de la percepción computacional, ya que permite mejorar las características visuales de una imagen y facilitar su posterior análisis mediante técnicas de procesamiento y visión por computador. Aspectos como el brillo, el contraste, la nitidez y la detección de bordes influyen en la forma en que se identifican los detalles, los contornos y las diferencias entre los objetos y el fondo de una escena. Por esta razón, comprender el comportamiento de estas características permite evaluar la calidad de una imagen y determinar qué transformaciones pueden contribuir a mejorar su interpretación.

Objetivos

Objetivo General

Evaluar y mejorar la calidad de imágenes reales mediante el uso de herramientas de OpenCV y métricas en Python, con el fin de diagnosticar problemas de captura (como iluminación, enfoque, contraste o perspectiva), aplicar un tratamiento justificado y comprobar su efectividad para optimizar el rendimiento en tareas de percepción computacional.

Informe de resultados de tratamiento de imagenes

Diagnóstico y comparación de métricas

Brillo es la media de intensidades en escala de grises; contraste se aproxima mediante desviación estándar; nitidez relativa se calcula como varianza del Laplaciano; resolución corresponde a ancho × alto. La nitidez no tiene un umbral universal y puede aumentar por textura del fondo, no solo por enfoque.

Imagen	Brillo	Contraste	Nitidez	Resolución	Diagnóstico
ANGULO.jpg	157.65	60.54	31.99	1,228,800	Brillo alto; nitidez inferior a la referencia.
DESENFOCADA.jpg	144.57	65.2	14.31	1,228,800	Nitidez baja, compatible con pérdida de enfoque.
FONDO_COMPLEJO.jpg	139.16	76.33	185.54	1,228,800	Contraste y Laplaciano altos; influyen texturas y bordes del fondo.
OSCURA.jpg	94.31	37.6	10.18	1,228,800	Menor brillo y contraste del conjunto.
REFERENCIA.jpg	148.88	58.92	139.93	1,228,800	Referencia de comparación; respuesta de bordes alta.

La imagen oscura tiene el brillo medio más bajo (94.31), contraste menor (37.60) y nitidez medida de 10.18. La desenfocada también registra nitidez baja (14.31), muy por debajo de 139.93 en la referencia. FONDO_COMPLEJO.jpg alcanza la mayor varianza del Laplaciano (185.54), pero esto no demuestra mejor reconocimiento: el fondo aporta bordes adicionales. Como todas las imágenes tienen la misma resolución, las diferencias métricas se relacionan con las condiciones de captura y el contenido.

Comparación JPG y PNG

Ambos formatos conservan 1,228,800 píxeles. PNG ocupa aproximadamente 8.9 veces el tamaño del JPEG, pero el error medio es cero en esta prueba. JPEG es eficiente para almacenamiento y transmisión; PNG conviene cuando se requiere conservar exactamente los valores de píxel o transparencia.

Formato	Resolución	Tamaño	Error medio	Interpretación
JPEG calidad 90	960 × 1280	119,952 bytes (117.1 KiB)	0.2603	Menor tamaño; compresión con pérdida, píxeles no idénticos.
PNG	960 × 1280	1,067,412 bytes (1,042.4 KiB)	0	Compresión sin pérdida; conserva exactamente los píxeles.

Tratamiento de la imagen problemática - Brillo y Contraste

La imagen presenta subexposición relativa. Aplicaré un ajuste lineal de brillo y contraste porque espero aumentar la visibilidad de regiones oscuras y mejorar la separación entre intensidades.

```
# Define los parámetros de la transformación.
factor_contraste = 1.6  # alpha mayor que 1 aumenta la separación entre intensidades.
ajuste_brillo = 10       # beta positivo suma intensidad y aclara la imagen.

# Aplica la transformación lineal: píxel_nuevo = alpha * píxel_original + beta.
# OpenCV limita el resultado al rango válido de una imagen de 8 bits (0 a 255).
img_ajustada = cv2.convertScaleAbs(
    img_tratamiento,
    alpha=factor_contraste,
    beta=ajuste_brillo
)

# Función que calcula las métricas requeridas para una imagen.
def calcular_metricas(imagen):
    """Devuelve brillo, contraste, nitidez y resolución de una imagen."""
    gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
    alto, ancho = gris.shape[:2]
    return {
        "Brillo promedio": gris.mean(),
        "Contraste (desviación estándar)": gris.std(),
        "Nitidez (varianza del Laplaciano)": cv2.Laplacian(gris, cv2.CV_64F).var(),
        "Resolución (píxeles)": int(ancho * alto)
    }
```

Imagen de referencia
Brillo: 148.88
Contraste: 58.92



Imagen oscura original
Brillo: 94.31
Contraste: 37.60

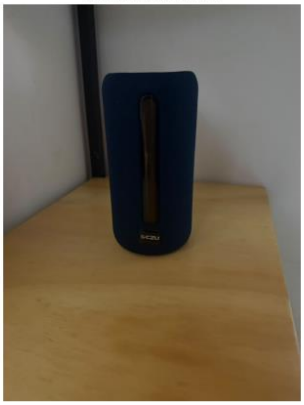
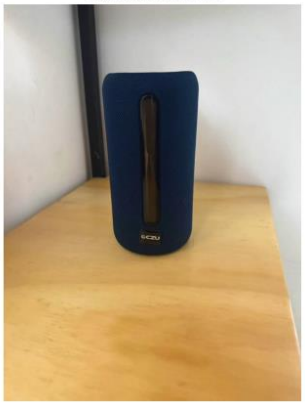


Imagen ajustada
Brillo: 160.89
Contraste: 60.18



Resultados cuantitativos antes y después

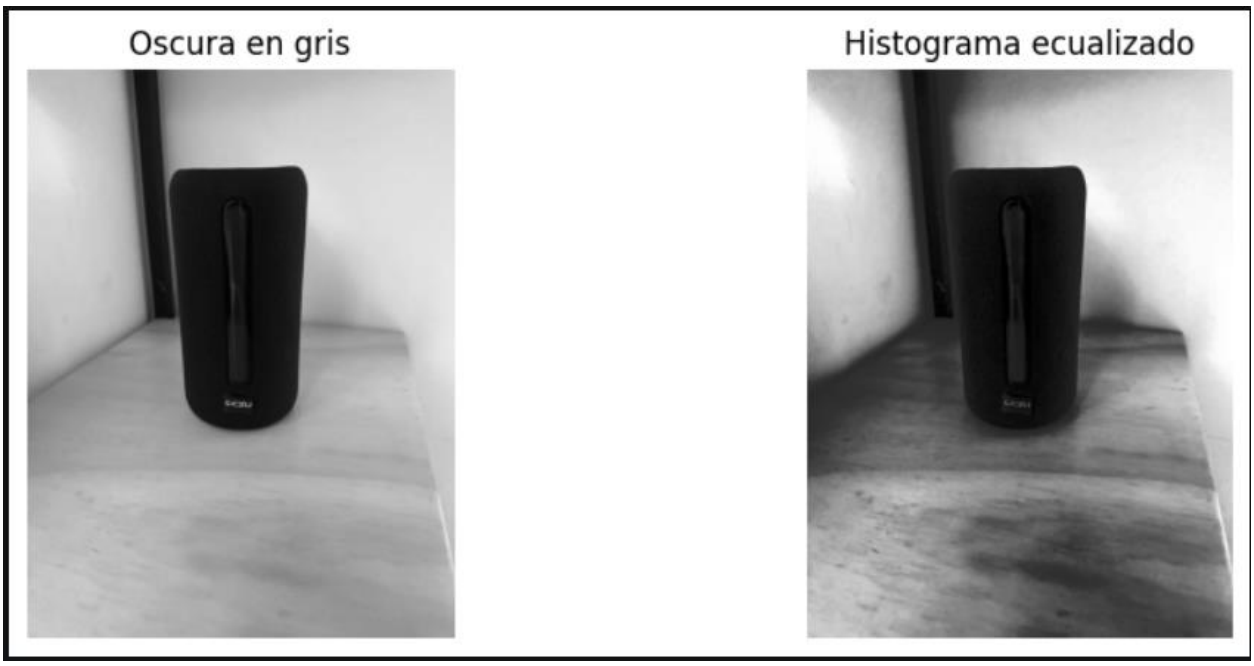
	Métrica	Antes	Después	Cambio (después - antes)
0	Brillo promedio	9.430572e+01	1.608926e+02	66.586915
1	Contraste (desviación estándar)	3.760162e+01	6.017833e+01	22.576710
2	Nitidez (varianza del Laplaciano)	1.018268e+01	2.659777e+01	16.415090
3	Resolución (píxeles)	1.228800e+06	1.228800e+06	0.000000

El brillo aumentó aproximadamente 70.6 % y el contraste 60.0 %. La resolución no cambió. La nitidez calculada también aumentó, pero no significa que se haya recuperado el enfoque: la transformación altera los gradientes y puede elevar la varianza del Laplaciano.

Con lo anterior logramos mayor visibilidad, el brillo ajusta la luminosidad general, ayudando a aclarar zonas oscuras o apuradas para que los detalles sean más fáciles de distinguir. Profundidad y riqueza en los tonos donde el contraste aumento la diferencia entre las áreas más claras y las más oscuras, lo que hace que los negros sean más profundos y los colores luzcan más vivos.

Tratamiento de la imagen problemática - Histograma

En esta sección se convierte la imagen oscura a escala de grises y se aplica la ecualización del histograma. La técnica redistribuye las intensidades para intentar mejorar el contraste global. Después se comparan visualmente la imagen original y la ecualizada.



En la imagen de referencia, la distribución de intensidades permite establecer un punto de comparación con las demás fotografías. En la imagen oscura, se espera una mayor concentración de píxeles hacia los niveles bajos de intensidad, lo que concuerda con el brillo promedio de 94,31, inferior al de la referencia (148,88). Esta distribución puede dificultar la identificación de bordes, texturas y características del objeto. Lo cual para el proximo ejemplo de detección de bordes no es lo más adecuada a utilizar.

Tratamiento de la imagen problemática - Detección del objeto

Para la detección del objeto se suavizan las imágenes con un filtro Gaussiano para reducir ruido y después se utiliza Canny para localizar cambios intensos de luminosidad. Los

bordes ayudan a estudiar los contornos, pero no constituyen por sí solos una máscara de extracción del objeto.

```
# Convertir la imagen de fondo complejo y la referencia a escala de grises.
# Así se analizan las variaciones de intensidad sin considerar los colores.
gris_compleja = cv2.cvtColor(img_compleja, cv2.COLOR_BGR2GRAY)
gris_referencia = cv2.cvtColor(img_referencia, cv2.COLOR_BGR2GRAY)

# Aplicar un desenfoque Gaussiano leve para reducir ruido antes de detectar bordes.
# El kernel (5, 5) define el tamaño de la ventana; sigma=0 deja que OpenCV lo calcule.
suavizada = cv2.GaussianBlur(gris_compleja, (5, 5), 0)
suavizada_referencia = cv2.GaussianBlur(gris_referencia, (5, 5), 0)

# Detectar bordes con Canny. Los umbrales determinan qué gradientes se consideran bordes.
# Valores distintos pueden producir cantidades diferentes de contornos.
bordes = cv2.Canny(suavizada, 50, 150)
bordes_referencia = cv2.Canny(suavizada_referencia, 100, 500)
```

Imagen de referencia



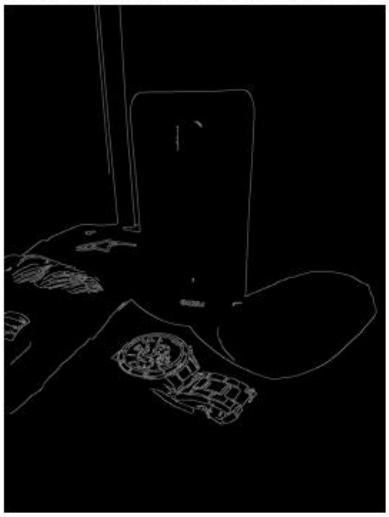
Bordes de la imagen de referencia



Imagen con fondo complejo



Bordes de la imagen con fondo complejo



identifica curvas en la imagen binaria de bordes y `boundingRect` calcula una caja rectangular alrededor de cada contorno. Las coordenadas están expresadas en píxeles y corresponden a la imagen cuyos bordes se analizaron. El filtro de área controla qué contornos se dibujan.

```python
# Buscar contornos externos en la imagen de bordes de referencia.
# RETR_EXTERNAL devuelve únicamente los contornos exteriores y CHAIN_APPROX_SIMPLE
# reduce los puntos almacenados para representar cada contorno.
contornos, _ = cv2.findContours(
    bordes_referencia, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

# Crear una copia de la imagen de referencia para dibujar sin modificar el original.
resultado = img_referencia.copy()

# Lista para registrar las áreas de los contornos que pasan el filtro.
areas = []

# Recorrer cada contorno detectado.
for c in contornos:
    # Calcular el área encerrada por el contorno, medida en píxeles cuadrados.
    area = cv2.contourArea(c)

    # Conservar contornos menores de 500 píxeles cuadrados.
    # Este umbral determina qué detalles se dibujan; ajústalo según el objeto y la imagen.
    if area < 500:
        # Obtener la caja delimitadora: x e y son la esquina superior izquierda;
        # w es el ancho y h es el alto del rectángulo.
        x, y, w, h = cv2.boundingRect(c)

        # Dibujar el rectángulo verde alrededor del contorno seleccionado.
        cv2.rectangle(resultado, (x, y), (x + w, y + h), (0, 255, 0), 2)

        # Guardar el área para poder revisar los contornos seleccionados.
        areas.append(area)
```



Ya con los contornos detectado y su respectiva coordenada del contorno mas granje correspondiente al rectángulo que encierra el parlante por completo x=319, y=214, ancho=313, alto=596, área=100 procedemos con GrabCut a intentar separar el primer plano del fondo a partir de un rectángulo inicial. La máscara resultante se utiliza como canal alfa para que el fondo sea transparente.



Como se observa en la imagen anterior la detección del objeto se logró en su gran mayoría, esto gracias al tratamiento previo de la imagen.

Respuestas a las preguntas orientadoras

¿Qué características de una imagen pueden afectar un sistema de percepción computacional?

La iluminación, el contraste, el enfoque, el ruido, la resolución, la perspectiva, la oclusión, el fondo y la compresión pueden modificar la información disponible. En esta práctica, la imagen oscura presenta menor brillo y contraste, y la desenfocada una varianza del Laplaciano reducida. Un fondo complejo agrega bordes que pueden confundirse con los del objeto.

¿Una imagen de mayor resolución necesariamente es mejor para un modelo?

No necesariamente. Más píxeles pueden conservar detalles finos, pero aumentan el costo computacional. Si la imagen está desenfocada, mal iluminada o el objeto ocupa pocos píxeles, aumentar las dimensiones no recupera la información perdida. La resolución debe ser suficiente para la tarea y compatible con la entrada del modelo todo dependiendo para que lo vamos a usar.

¿Qué efecto tuvo la iluminación sobre los datos obtenidos?

OSCURA.jpg tuvo brillo medio de 94.31 frente a 148.88 en la referencia, y contraste de 37.60 frente a 58.92. Esto concuerda con menor visibilidad y separación de intensidades. Tras el ajuste, el brillo llegó a 160.89 y el contraste a 60.18, evidencia numérica coherente con aclarar la imagen.

¿Qué ocurrió con la imagen desenfocada y cómo se reflejó en la medida de nitidez?

DESENFOCADA.jpg obtuvo nitidez de 14.31, muy inferior a 139.93 frente a la imagen de referencia. El desenfoque suaviza los cambios rápidos de intensidad y reduce la respuesta del Laplaciano. La métrica es comparativa y también vario por textura o ruido.

¿Todos los problemas encontrados pueden corregirse mediante procesamiento?

No. El ajuste de brillo puede hacer más visible la información capturada y la segmentación puede reducir la influencia del fondo, pero el procesamiento no reconstruye con certeza detalles eliminados por desenfoque intenso, saturación u oclusión. Algunos defectos requieren volver a capturar la imagen con mejor enfoque, iluminación o ángulo para un mejor procesamiento.

¿Por qué es importante analizar una imagen antes de utilizarla como entrada de un modelo?

El diagnóstico permite detectar entradas inadecuadas, definir un preprocesamiento consistente y evitar que diferencias de captura se confundan con diferencias reales entre objetos. También ayuda a decidir si conviene corregir, normalizar, segmentar o volver a capturar la muestra. Sin ese análisis, el modelo puede aprender patrones accidentales del fondo o la iluminación o patrones incorrectos afectando directamente los resultados finales.

¿El tratamiento aplicado mejoró realmente la imagen para el propósito planteado?

Para el objetivo acotado de mejorar la visibilidad de una imagen subexpuesta, sí hay evidencia favorable: brillo de 94.31 a 160.89 y contraste de 37.60 a 60.18; la resolución se mantuvo en 1,228,800 píxeles. El aumento de nitidez medida (10.18 a 26.60) no prueba recuperación de detalles. Se demuestra mejora fotométrica, no mayor precisión de un modelo.

Desde el análisis de la detección de bordes, considero que el tratamiento aplicado a la imagen oscura sí generó una mejora potencial para identificar los contornos y las estructuras

presentes en la escena. Al aumentar el brillo y el contraste mediante la transformación de intensidad, se hicieron más visibles algunas diferencias entre las regiones de la imagen, lo que puede facilitar la identificación de transiciones entre objetos y fondo.

Anexo

A continuación, comparto el link del repositorio donde se encuentra alojado el codido de este trabajo.

https://github.com/edwardmoreno18/PERCEPCION_COMPUTACIONAL

Conclusiones

A partir de los resultados obtenidos, concluyo que el tratamiento aplicado a la imagen oscura mejoró sus condiciones visuales al aumentar el brillo promedio de 94,31 a 160,89 y el contraste de 37,60 a 60,18. Estos cambios permitieron obtener una imagen con mayor luminosidad y una diferenciación más evidente entre algunas de sus regiones, facilitando potencialmente la identificación de los objetos y sus contornos.

En cuanto a la detección de bordes, la varianza del Laplaciano aumentó de 10,18 a 26,60, lo que indica una mayor respuesta a los cambios locales de intensidad. Este resultado sugiere que el tratamiento puede favorecer la identificación de contornos; sin embargo, no garantiza por sí mismo una detección más precisa, debido a que el incremento del contraste también puede resaltar el ruido o producir bordes no correspondientes a objetos reales.